COMS 4130 Project Status Update 3

by Taylor Bauer, Abby Van Der Brink, Trevor List

COMPLETED

CFG & ICFG (Taylor)

- initially generated full CFGs using ast for the 4 functions but since it is a big program, there was a horizontal rendering issue and could not figure out how to fix
- switched to making 2 ICFG for beamforming pipeline with stream_to_array_data, run_fk, beam_window, compute_beam_power, find_peaks and detection pipeline with detect_signals, run_fd, calc_det_thresh
- run_fd is too complex/too many lines (it has like 50+ nodes) for its own CFG so it is in the icfg for detection instead

Dataflow Analysis (Taylor)

- created python script using ast to track important variables, e.g. (fstat_vals, det_mask, thresh, beam_peaks, etc…) in the functions run_fd, calc_det_thresh, run_fk, find_peaks, detect_signals, beam_window
- gave dataflow.json to abby for her queries

Coverage (Taylor)

- ran coverage script on test_beamforming.py
- coverage.py was having some compatibility issues but a fix has been found and coverage.json has been given to abby with

Call Graph Query Model (Abby)

- Created a call_graph_query.py module that implements BFS and reverse BFS traversal over the call graph edges stored in static_analysis table
- Includes an interactive shell
- **Supported queries:**

```
Commands:
  is_caller <A> <B>     -- Is A a caller of B?
  path <A> <B>          -- Shortest call path from A to B
  callees <A>           -- All functions A can reach
  callers <B>           -- All functions that call B
  direct <A>            -- Direct calls made by A
  who_calls <B>         -- Direct callers of B
  list                  -- List all functions in graph
  quit                  -- Exit
========================================

query>
```

- **Verified Query Results:**

```
query> is_caller detect_signals calc_det_thresh

 Yes! detect_signals is a caller of calc_det_thresh
 Path:
 detect_signals-> run_fd-> calc_det_thresh
```

```
query> callees beam_window

 All functions reachable from beam_window (8 total):
 * build_slowness
 * compute_beam_power
 * fft_array_data
 * find_peaks
 * project_ABA
 * project_ABc
 * project_Ab
 * run
```

```
 query> callers calc_det_thresh

  All functions that call calc_det_thresh (2 total):
  * detect_signals
  * run_fd
```

```
query> list

 All 17 functions:
 * beam_window
 * beam_window_wrapper
 * build_slowness
 * calc_det_thresh
 * compute_beam_power
 * compute_beam_power_wrapper
 * compute_delays
 * detect_signals
 * fft_array_data
 * find_peaks
 * project_ABA
 * project_ABc
 * project_Ab
 * run
 * run_fd
 * run_fk
 * stream_to_array_data
```

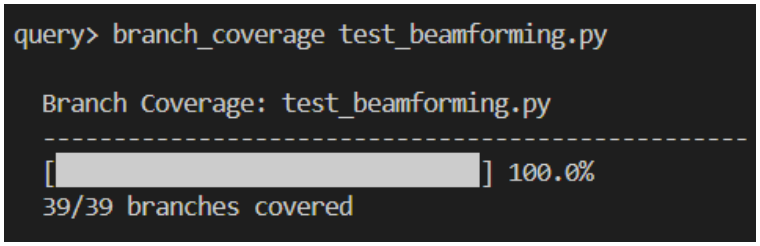
Coverage Query Model (Abby)

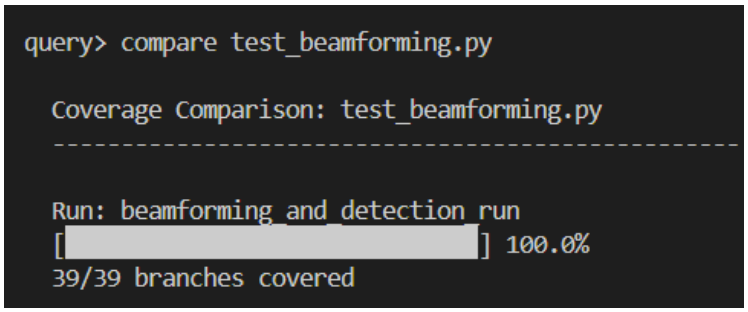
- Created coverage_query.py module to answer SQL-based coverage and dataflow dependency questions
- Includes an interactive shell
- For both query modules i used infraPy (installed with –no-deps)
- **Supported Queries:**

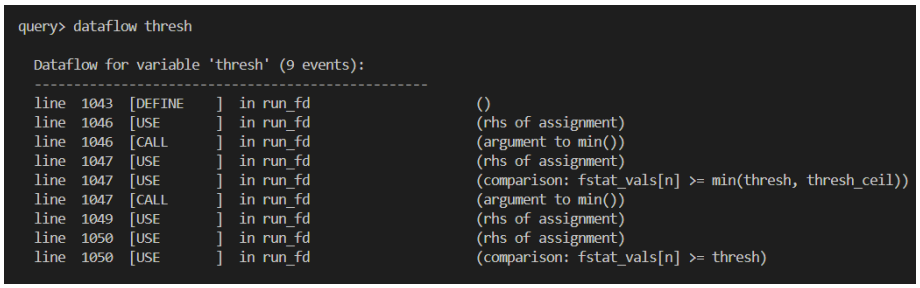
```
Commands:
  branch_coverage <file>       -- Coverage % and uncovered branches
  uncovered <file>             -- All uncovered line numbers
  compare <file>               -- Compare across test runs
  dataflow <var>               -- Where is a variable used?
  dependent <var_a> <var_b>    -- Are two variables data-dependent?
  summary                      -- Overall coverage across all files
  quit                         -- Exit
```

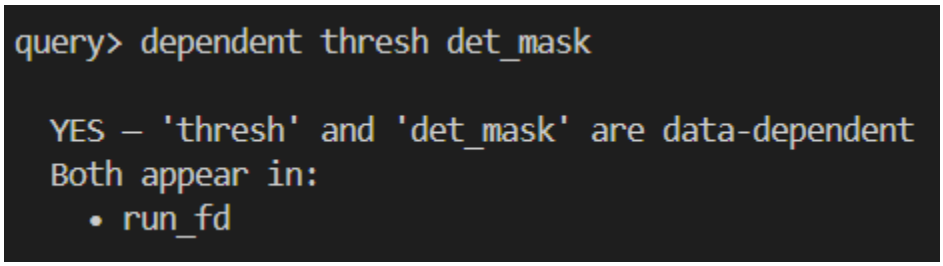
- **Verified Query Results:**

```
query> summary

  Overall Coverage Summary
  ------------------------------------------------

  _spectral.py
  [                                ] 100.0%
  8/8 branches covered

  beamforming_new.py
  [                                ] 52.2%
  303/581 branches covered

  spectral.py
  [                                ] 2.6%
  4/156 branches covered

  test_beamforming.py
  [                                ] 100.0%
  39/39 branches covered

query> branch_coverage test_beamforming.py

  Branch Coverage: test_beamforming.py
  ------------------------------------------------
  [                                ] 100.0%
  39/39 branches covered
```









Abstract Interpretation and Fuzzing (Trevor)

- Reviewed the Adaptive Fisher Detector logic
- Identified key input parameters for analysis
- Began outlining abstract interpretation rules
- Set up initial groundwork for fuzzing using sample soundwave data

Some Future Work/Next Steps For Final Project

Taylor: Start the presentation this week

Abby:

- Work with the team to create a presentation
- Show off interactive shell for both query systems in presentation

- Create a keyword router to wire both query modules into one dispatcher

Trevor:

- Finalize abstract interpretation rules and apply them to identify potential incorrect detector behaviors
- Expand fuzzing experiments with more diverse and extreme input cases
- Export fuzzing and abstract interpretation results into a format compatible with the team's database
- Assist with integration and validation of all analysis data
- Contribute to presentation materials